BUILD THESE NEXT, IN THIS ORDER (continues from step 41)

Steps 42 to 45 are a warm-up. Steps 46 to 60 are the core. Steps 61 to 75 are the hard part (plan about 3 days). Steps 76 to 78 are stretch. Rules as before: Flask and the standard library only, integer paise for money, ? placeholders in SQL, and the server checks ownership. Back up your .db file before every step that adds a table or column, and use a migration for it.

PART A: DISCOVERY (easier, good warm-up)

42. Recently viewed books

Keep the last 5 viewed book ids in the session and show them on the home page.

Done when: the same book viewed twice appears once, and a deleted book never crashes the list.

43. Author pages

Click an author name to open /author/<name> with all their books and a count.

Done when: an unknown author shows your 404 page, and the page uses one query.

44. New arrivals and Coming soon

Add release_date to books. Books released in the last 14 days get a "New" ribbon. Future books show "Coming soon" with no Add to Cart button.

Done when: the server refuses an order for a future book even if the request is forced.

45. Compare two books

Tick two books and open /compare to see price, author, rating and stock side by side.

Done when: choosing 1 or 3 books shows a clear message, not an error.

PART B: SELLING MORE

46. Combo bundles

Admin creates a bundle (several books, one price in paise). The customer adds the whole bundle to the cart.

Done when: buying a bundle reduces the stock of every book in it, and it fails completely if any one book is out of stock.

47. Flash sale with a timer

A book has sale_price_paise, sale_start and sale_end. The sale price applies automatically only between those times.

Done when: no cron or manual action is needed, and the cart, the order and the invoice all use the same price.

48. Gift option

A checkbox at checkout for gift wrap (extra Rs 30) and a message of up to 150 characters. Show it on the invoice.

Done when: the message is escaped on the page, so <script> typed in it appears as plain text.

49. Loyalty points

1 point per Rs 100 spent, and 100 points are worth Rs 10 off. Keep a points_ledger table and calculate the balance with SUM.

Done when: cancelling or returning an order removes the points it earned.

PART C: COMMUNITY

50. Review helpful votes

A "Helpful" button on each review, one vote per user per review, shown as a count.

Done when: a double click creates one vote, protected by a UNIQUE pair.

51. Report a review (moderation)

A user can report a review. The admin sees reported reviews and can hide or keep them.

Done when: a hidden review disappears from the page. Decide whether its rating still counts in the average, and write your reason in a comment.

52. Request a book

A form where a customer asks for a book you do not stock. The admin sees requests ordered by how many people asked for the same title.

Done when: two people asking for the same title increases a count instead of creating a duplicate row.

53. Share my wishlist

The user generates a public link /w/<token> that shows a read-only wishlist. They can turn the link off.

Done when: the token is random and long, and a turned-off link shows 404.

PART D: STORE OWNER TOOLS

54. Price history

Every price change writes a row to price_history (book_id, old, new, changed_by, changed_at). Show a small table on the admin book page.

Done when: the history is written automatically from ONE function, not copied into each route.

55. Bulk price change

On the admin books list, tick several books and apply "increase by 10%" or "set category discount".

Done when: all selected books change in one transaction, and rounding is correct to the paise.

56. Dashboard charts (CSS only)

Show a bar chart of the last 7 days of sales using plain divs with a percentage height.

Done when: it works with no JavaScript and no library, and an empty day shows an empty bar, not a gap.

57. Staff role with limited rights

Add a "staff" role who can pack and ship orders but cannot touch coupons, prices or reports.

Done when: staff typing an admin URL gets 403, using your existing role decorator.

PART E: USER CONTROL

58. Download my data

A button that downloads the user's profile and orders as a JSON file.

Done when: it contains only that user's data.

59. Delete my account

Ask for the password, then anonymise the user (name becomes "Deleted user", email cleared) but keep the orders for accounting.

Done when: old invoices still open, and the person can no longer log in.

60. Dark mode toggle

A button in the header that switches the CSS variables in your style.css and remembers the choice in localStorage.

Done when: no page flashes white on load.

PART F: HARD ORDERS AND MONEY

61. Idempotent checkout

Put a one-time checkout token (random, stored in a table with a UNIQUE column) in the checkout form. Placing the order marks the token used.

Done when: double-click, refresh after submit and the browser Back button all create exactly ONE order, and the second attempt shows the existing order instead of an error.

62. Edit an order before packing

While the status is Placed, the customer can change quantities or remove an item. Stock, coupon, shipping, tax and total are all recalculated in one transaction.

Done when: a coupon with a minimum order amount is re-checked after the edit and removed with a clear message if it no longer qualifies, and the paise still add up.

63. Pricing rules engine

Table pricing_rules (type, book_or_category, value, priority, active). Support "Buy 2 get 1 free" and "X% off a category". Apply them in priority order from ONE function.

Done when: the cart, order and invoice show each applied rule per line, and changing a priority changes the result without touching any route.

64. Gift cards

A customer buys a gift card and gets a code. Store only a HASH of the code. Another customer redeems it at checkout, fully or partly. The balance is the SUM of a ledger table.

Done when: two browser tabs cannot spend the same balance twice, and an unknown, used-up or expired code each shows its own message.

65. Backorders with a fair queue

An out-of-stock book can be ordered as a backorder. When the admin receives stock, waiting orders are fulfilled oldest first, automatically, in one transaction.

Done when: two restocks serve customers in the order they waited, and no copy is ever sold twice.

PART G: DATA AND CONCURRENCY

66. Optimistic locking on admin edits

Add a version column to books. The edit form carries the version it loaded. Saving succeeds only if the version is unchanged, then increases it.

Done when: two admins editing the same book in two tabs: the second save is refused with a clear message, and no change is lost silently.

67. Keyset pagination for the API

Add /api/books?after=<id>&limit=10 using WHERE id > ? ORDER BY id LIMIT ?, and return a next_cursor in the JSON.

Done when: adding or deleting books while someone pages through never causes a skipped or repeated book, and limit is capped at 50.

68. Tamper-evident audit log

Each admin_log row stores a hash of (previous row hash + its own content). Add an admin page "Verify log".

Done when: editing or deleting one old row by hand in the db makes the verify page show exactly which row broke the chain.

69. Notification outbox with retry

When an event happens (order shipped, return decided), write a row to an outbox table in the SAME transaction. A function sends pending rows (print to console), counting attempts, and gives up after 3 failures.

Done when: a simulated failure is retried, a sent message is never sent twice, and a failed row stays visible to the admin.

70. Book of the month subscription

A customer subscribes to a category and gets one new order every month. A "Run due subscriptions" admin button creates the due orders. No cron.

Done when: pressing the button twice in a row creates the orders only once, and a customer with no stock or no address gets a skipped row with a reason.

PART H: ADVANCED SECURITY

71. Two-factor login (TOTP, standard library only)

Implement the 6-digit time-based code yourself with hmac, hashlib, struct and time (RFC 6238). Show a secret as text, accept the code from any authenticator app.

Done when: a code is accepted within 1 step before or after the current time, the same code cannot be used twice, and the secret is never shown again after setup.

72. API keys with scopes

A user creates an API key with a scope (read_books or read_orders). Show the key ONCE, store only its HASH. Check the key in a decorator on /api/ routes.

Done when: a key without the scope gets 403 in JSON, a deleted key stops working at once, and the rate limit from Part 4 is counted per key.

73. Permissions table (replace role checks)

Tables permissions and role_permissions. Write a permission_required("coupons.edit") decorator. Admin can change what staff may do from a page.

Done when: giving or removing a permission changes access on the next click without a code change, and every old @role_required is replaced.

PART I: ANALYTICS

74. Growth report with window functions

Page showing monthly revenue, change from the previous month in percent, and a running total, using SQL window functions (LAG, SUM OVER).

Done when: the first month shows no percentage instead of crashing, and the figures match a hand calculation from the CSV export.

75. Recommendations for me

On the home page show books a customer has not bought, scored by how often other customers who bought their books also bought them. One SQL query with JOIN and GROUP BY.

Done when: a new user with no orders sees bestsellers instead, and an already owned book never appears.

STRETCH STEPS (only after 42 to 75 all work)

76. Automated test suite

Use unittest and the Flask test client with a temporary database created by migrate.py. Write at least 20 tests covering money totals, coupon rules, the last-copy checkout, a 403 on another user's order and the idempotent checkout.

Done when: one command runs everything, and removing one ownership check or one UNIQUE constraint makes a test fail.

77. Concurrency stress script

Write stress.py that starts 20 threads, each trying to buy the last copy at the same moment.

Done when: exactly one order succeeds, stock never goes below 0, and the script prints the result.

78. Final defence, round 3

Demo on a fresh database created only by migrate.py, with the tests passing. Each trainee explains one idempotency protection, one ledger or hash calculation and one permission check from their own code, with no notes. Then 2 live changes (examples: "Buy 2 get 1 free becomes Buy 3 get 1 free", "limit API keys to 3 per user", "add a read_wishlist scope").

Done when: the live change is working in under 15 minutes.

QUESTIONS

- Why does a sale price need to be copied into the order line instead of read from the book later?
- Why is a UNIQUE pair better than checking in Python for helpful votes?

- Why must a public share link use a long random token and not the user id?
- Why do we anonymise a deleted user instead of deleting the row?
- Why must a bundle purchase be one transaction?
- What problem does an idempotency token solve, and why must the database enforce it?
- Why does the optimistic lock refuse the second save instead of letting the last one win?
- Why do keyset pages stay correct when rows are added, but OFFSET pages do not?
- Why does a hash chain show tampering, and what can it NOT protect against?
- Why write the outbox row in the same transaction as the order change?
- Why does TOTP accept a window of codes, and why must a used code be refused?
- Why is an API key shown once, and what is stored instead?
- What does a permissions table give you that a role name inside each route does not?

SUGGESTED SPLIT FOR TRAINEES

Slower trainee	42, 43, 44, 48, 50, 60
Average trainee	add 45, 47, 52, 53, 54, 58, 61, 66
Strongest trainee	add 46, 49, 55, 57, 59, 62 to 65, 67 to 75, and the stretch steps 76 to 78

DO NOT

- Start the next step before this one works
- Use float for money anywhere
- Build SQL strings by joining user input
- Trust an id in the URL without checking who owns it
- Copy code you cannot explain line by line
- Store a key, code or token in a form that can be used if the table is read
- Rely on a Python check where the database can enforce the rule
- Install extra libraries (the 2FA code must use only the standard library)